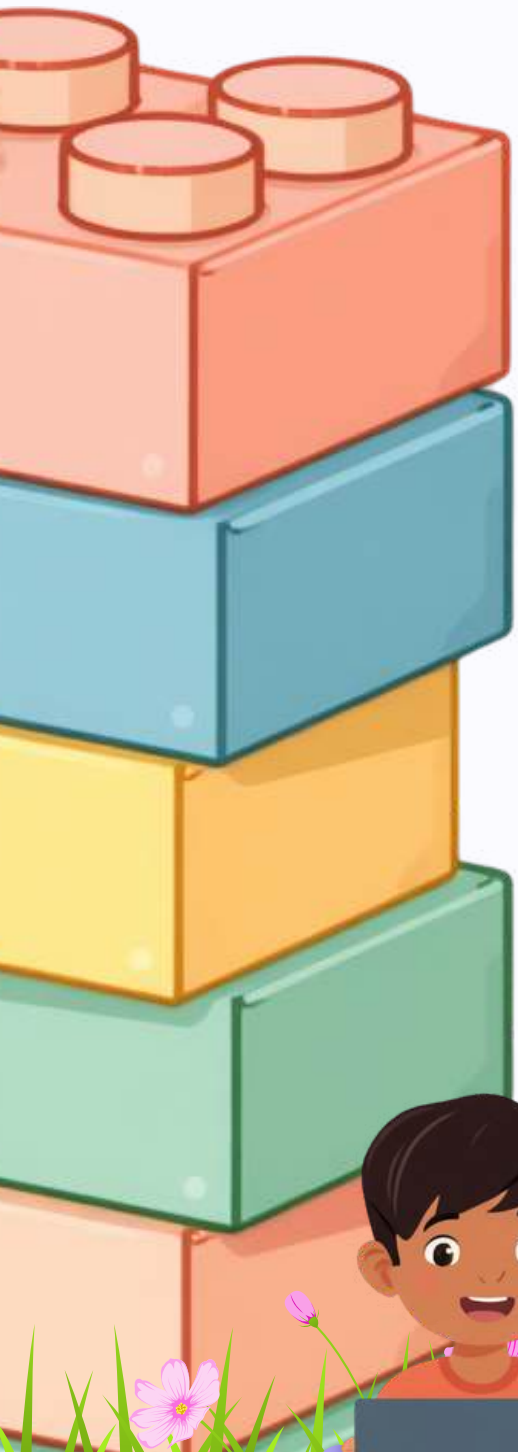




TYPESCRIPT

INTERFACES & TYPE ALIASES

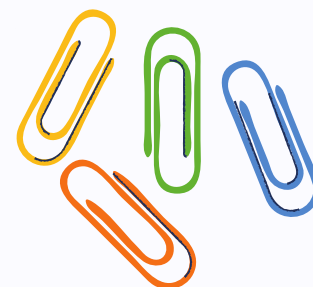
THE RULEBOOKS OF CODE





Messy Digital Library Challenge

Imagine you have a huge digital library but all the books are mixed up and messy!
How can we organize our digital world like a clean, neat cupboard?
Today we will learn the tools that help us keep our code organized and tidy!

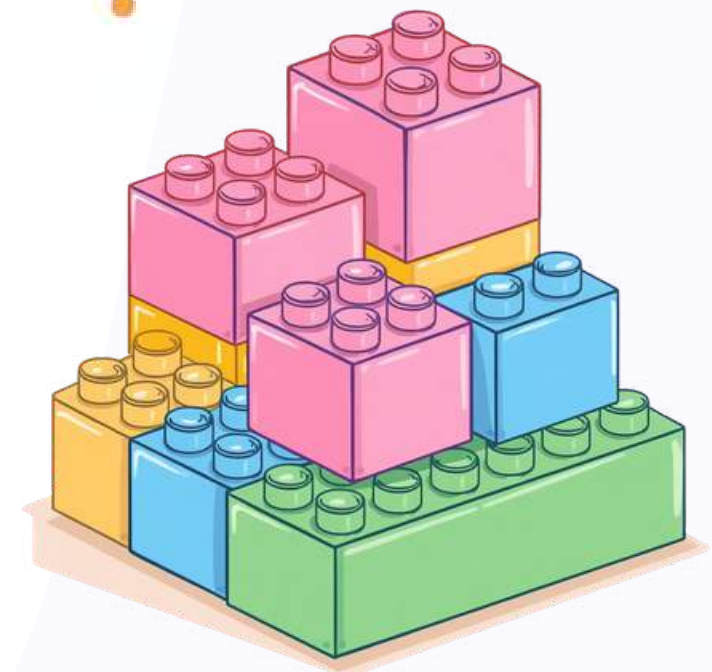
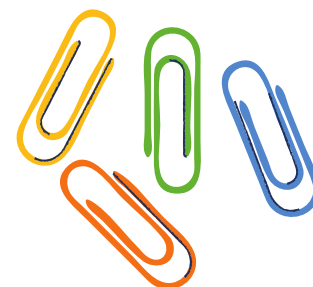




Quick Recap

We learned about strings, numbers, and booleans — the simple building blocks of code. These basic types help TypeScript understand what kind of data we are working with!

- ✓ Name: string — stores text like "Priya" or "Arjun"
- ✓ Age: number — stores numbers like 11 or 12



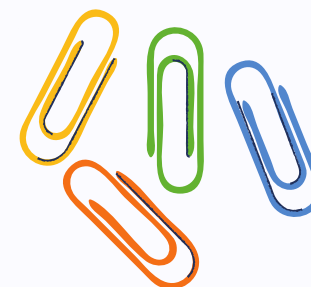


What is an Interface?

An Interface is like a blueprint or a rulebook for an object. Think of it like an Aadhaar card — every Aadhaar card must have a Name, Address, and a 12-digit Number. If any of these are missing, the card is incomplete!

- ✔ Interface = Rules for what information must be there
- ✔ If a property is missing, TypeScript will tell you!

The features of Aadhaar are the properties for a citizen!

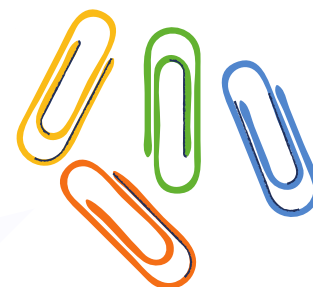




Interface Example: Student Profile Rulebook

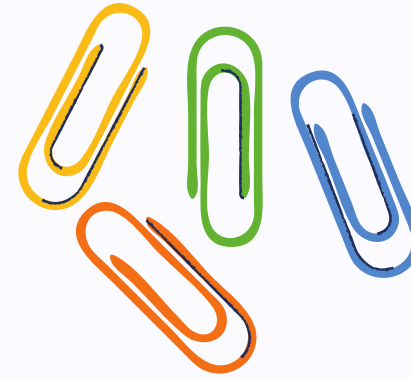
```
interface Student {  
  name: string;  
  age: number;  
  grade: number;  
}
```

This is the "rulebook" for a Student — it tells TypeScript exactly what properties every student object must have!





Code Comparison



Messy Code ✗

```
let student1 = {  
  nam: "Priya",  
  age: "twelve"  
}
```

Missing grade, wrong types!

Organized Code ✓

```
let student1: Student = {  
  name: "Priya",  
  age: 12,  
  grade: 6  
}
```

Interface keeps it correct!

Key Takeaway 💡

Interfaces help keep our code neat and correct — like organizing your school cupboard!



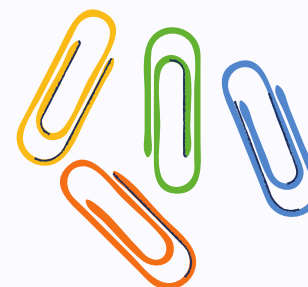


Mandatory vs Optional

Some information is mandatory — like Student Name and Parent Phone Number. These fields **MUST** be filled in!

Other information can be optional — like Email ID. You can skip it if you don't have one.

- ✔ `studentName`: string — Required field, must be filled!
- ✔ `emailID?`: string — Optional field, the ? means it's okay to skip!





What is a Type Alias?

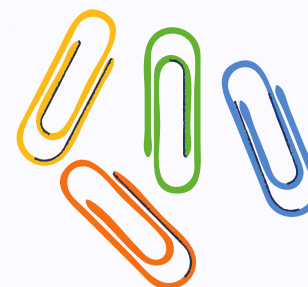
A Type Alias is like a pet name or nickname for a type!

Instead of writing "a sequence of numbers representing a pincode," we simply call it Pincode.

Code Example:

```
type Pincode = string;
```

Now whenever we need a pincode, we just use our nickname "Pincode" instead of explaining it every time!





Type Aliases

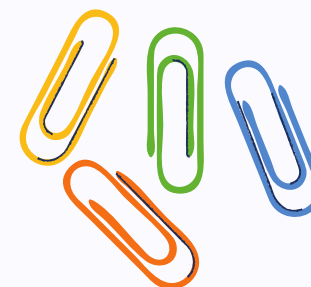
A Type Alias with Union Types lets one nickname represent multiple possible types. This makes our code flexible and easy to read!

Code Example:

```
type ID = number | string;
```

This means an ID can be either a number (like 12345) or a string (like "STU-001").

- ✓ The nickname "ID" makes code simple and clear to understand.
- ✓ Union types use the | symbol to combine multiple type options.

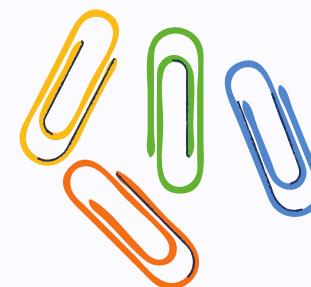




Cricket Player Stats Interface Combined!

```
interface CricketPlayer {  
  name: string;  
  runs: number;  
  wickets?: number;  
  team: string;  
  playerID: ID;  
}
```

We can organize cricket player data using interfaces and type alias nicknames together! The playerID uses our ID alias.

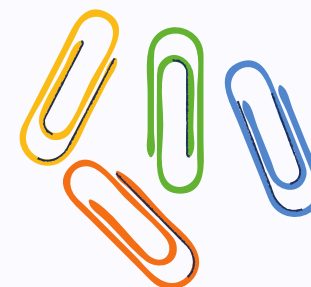


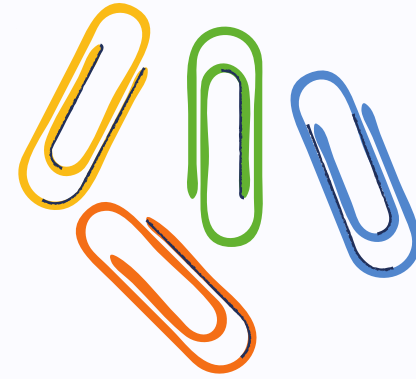


Activity: Build Your Own Interface!

Design an interface for a Cricket Player Card or a Zomato Order Summary. Think about what information each one needs!

Cricket Player Card: Name (string), Runs (number), Wickets (optional number), Team (string). Zomato Order: Dish Name (string), Price (number), IsVeg (boolean).





When to Use Each One?

Interfaces

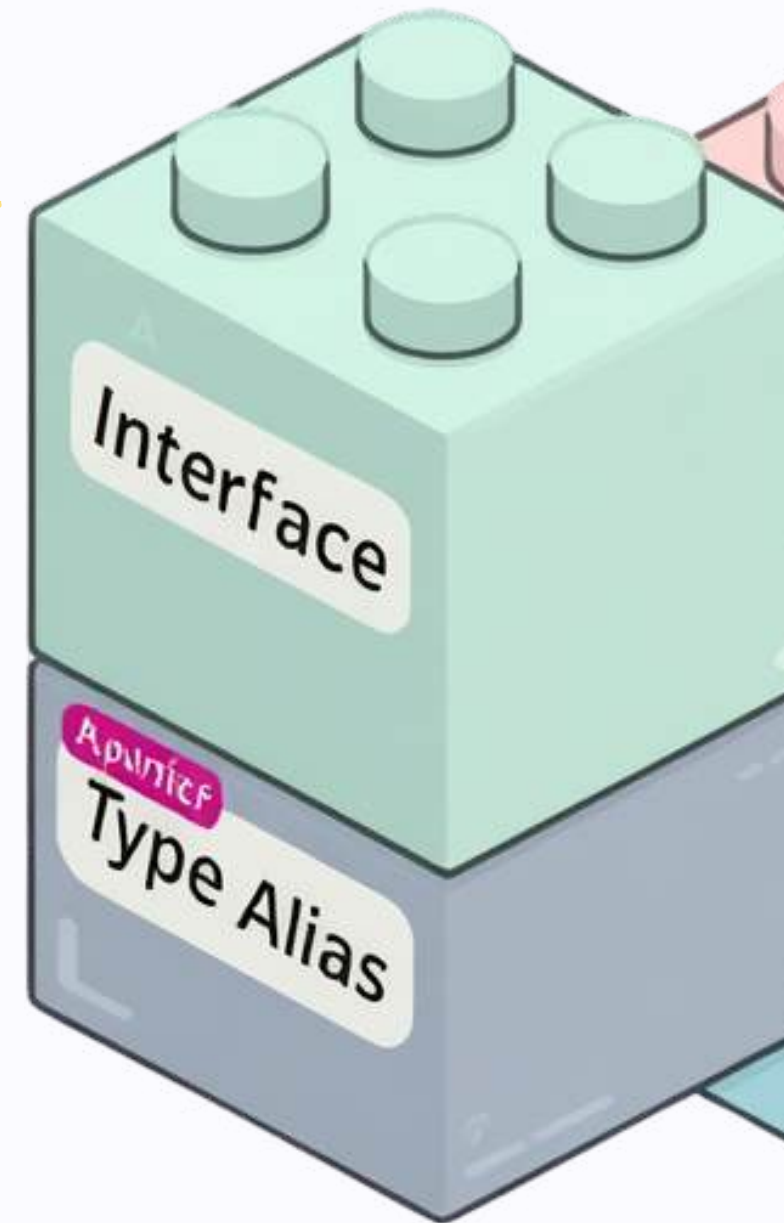
Use interfaces to create the blueprint or rulebook for objects. They define the structure!

Type Aliases

Use type aliases as nicknames for simple or complex types. They simplify names!

Together

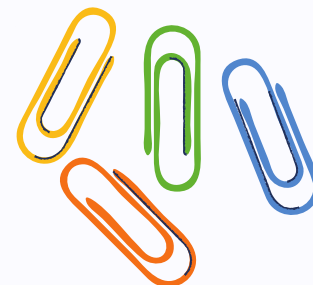
Interfaces define structure, Aliases simplify type names. They work as a team!





Key Takeaways Remember These Important Points!

- Interfaces = Rulebooks/Blueprints for data objects — they define what properties must be there!
- Type Aliases = Nicknames for types — making code easier to read and write.
- Optional properties are like optional fields on forms — use ? to mark them.
- Organizing code keeps it neat, just like organizing your school cupboard!





THANK YOU!

Ready to Build Your Own Code Rulebooks?

Thank you for learning with us!
Planrs Academy

